

```

# CANONICAL QUADRATIC ITERATOR IMPLEMENTATION
# Phillip Somerville (@unitambo) - paste at start of new session

import math
from collections import Counter, defaultdict

# --- PARAMETERS ---
C_STATIC = 1.094          # static F value (bit=0)
DELTA     = 0.002          # delta (bit=1 → F=1.096)
PREC      = 14             # decimal precision
EPS       = 5e-15          # half ULP at 14dp

# --- CORE TRUNCATION ---
def trunc_round(x, dp=PREC):
    """TRUNC(ROUND(x, 14), 14) - the precision discipline."""
    r = round(x, dp)
    f = 10**dp
    return math.floor(r * f) / f if r >= 0 else math.ceil(r * f) / f

# --- SINGLE PASS ---
def encode_node(payload, seed=0.087):
    """
    Single pass of iterator over payload from seed.
    Returns (digest, control_stream, depth).

    payload:  string - any UTF-8
    seed:     float  - starting H value (nonce)

    Returns:
    digest:    float  - terminal H value
    B:        list   - ternary control stream {-1, 0, +1}
    N:        int    - total steps (8*len(payload) + 2)
    """
    bits = [int(x) for ch in payload for x in format(ord(ch), '08b')]
    b_seq = bits + [0, 0]                # two closing zeros
    N      = len(b_seq)
    F_seq = [C_STATIC + (DELTA if b == 1 else 0) for b in b_seq]

    H = seed
    B = []

    for n in range(N):
        H_new = trunc_round((H**2 - F_seq[n])**2 - F_seq[n])

```

```

        # Inverse interrogation - ternary decision
        inner = H_new + F_seq[n]
        j = 0
        if inner >= 0 and F_seq[n] >= math.sqrt(inner):
            k = round(math.sqrt(F_seq[n] - math.sqrt(inner)), PREC)
            j = round((k - H) * 1e14)

        B.append(j)
        H = H_new

    return H, B, N

# — FIXED POINT (D*) —————
def fixed_point(payload, seed=0.087, max_loops=20):
    """
    Find canonical fixed point D* for payload.
    Seed-independent - converges in 2 loops.
    D* satisfies: encode_node(payload, D*)[0] == D*
    """
    D = seed
    for _ in range(max_loops):
        D_new, _, _ = encode_node(payload, D)
        if abs(D_new - D) < 1e-14:
            return D_new
        D = D_new
    return D

# — BUILD CHAINED INDEX —————
def build_index(lines):
    """
    Build chained digest index from list of (id, payload) pairs.
    Each digest feeds as seed to the next node.

    Returns dict: {node_id: {payload, digest, depth, seed, B, overhead}}
    """
    idx = {}
    seed = 0.087

    for node_id, payload in lines:
        digest, B, N = encode_node(payload, seed)
        nonzero = sum(1 for j in B if j != 0)
        idx[node_id] = {
            'payload': payload,
            'digest': digest,
            'depth': N,
            'seed': seed,
            'B': B,

```

```

        'overhead': nonzero / N,
    }
    seed = digest

    return idx

# — D* DISTANCE METRIC —————
def d_star(p1, p2):
    """
    D* distance between two payloads.
    Satisfies identity, symmetry, triangle inequality.
    """
    return abs(fixed_point(p1) - fixed_point(p2))

# — QUAD CHANNEL —————
def quad_channel(text):
    """
    Four canonical channel fingerprint.
    forward, reverse, complement, reverse complement.
    """
    def get_bits(t):
        bits = [int(x) for ch in t for x in format(ord(ch), '08b')]
        return bits + [0, 0]

    bits_f = get_bits(text)
    bits_r = get_bits(text[::-1])
    bits_c = [1 - b for b in bits_f]
    bits_rc = [1 - b for b in bits_r]

    def run(bits):
        F_seq = [C_STATIC + (DELTA if b == 1 else 0) for b in bits]
        D = 0.087
        for _ in range(20):
            H = D
            for F in F_seq:
                H = trunc_round((H**2 - F)**2 - F)
            if abs(H - D) < 1e-14:
                return H
        D = H
        return H

    return {
        'forward': run(bits_f),
        'reverse': run(bits_r),
        'comp': run(bits_c),
        'revcomp': run(bits_rc),
    }

```

```

# — CLUSTERING STATS —————
def clustering_rate(idx):
    """Compute D* clustering rate for an index."""
    exact_freq = defaultdict(list)
    for pid, e in idx.items():
        exact_freq[e['digest']].append(pid)
    clustered = sum(len(v) for v in exact_freq.values() if len(v) > 1)
    return 100 * clustered / len(idx), exact_freq

# — MUSICAL ENCODING CONVENTION —————
# Naturals: C D E F G A B
# Flats:    Cb Db Eb Fb Gb Ab Bb
# Sharps:   Cs Ds Es Fs Gs As Bs
# Concatenate note names directly — no separators
# e.g. D-flat major opening of Clair de Lune:
#      "AbAbAbFEbDb"
# G major (Bach) with F#:
#      "GDBGDBFs"

# — QUICK TEST —————
if __name__ == "__main__":
    # Verify construction is working correctly
    print("Canonical iterator — self-test")
    print()

    # Fixed point test
    payload = "the rain in Spain falls mainly on the plain"
    D = fixed_point(payload)
    D_check, _, _ = encode_node(payload, D)
    print(f"Payload: '{payload}'")
    print(f"D*:      {D:.14f}")
    print(f"Self-certifies: {abs(D_check - D) < 1e-14} ✓")
    print()

    # Rhyme detection
    pairs = [("rain", "Spain"), ("rain", "plain"), ("cat", "rat")]
    print("Rhyme detection (forward channel):")
    for p1, p2 in pairs:
        d = d_star(p1, p2)
        print(f"  {p1}/{p2}: {d:.6e} {'← rhyme' if d < 1e-10 else ''}")
    print()

    # Palindrome identity
    palindromes = ["racecar", "level", "madam"]
    print("Palindrome identity KL(f,r) — D* forward vs reverse:")
    for p in palindromes:

```

```
q = quad_channel(p)
diff = abs(q['forward'] - q['reverse'])
print(f"    {p}: |D*f - D*r| = {diff:.6e} {'✓' if diff < 1e-10 else ''}")
```